

Customer Support Ticket Analyzer using Python

1. Project Overview and Objective

1.1 Project Overview

This project involves developing a Python-based customer support ticket analysis system. The project uses a dictionary of lists to store ticket information and applies Python programming concepts to add, clean, process, and analyze customer support issues.

The workflow covers ticket data loading, ticket addition and validation, text cleaning, keyword analysis, priority analysis, longest-issue identification, unique-word extraction, and final report generation.

1.2 Project Objective

The main objective of this project is to demonstrate practical Python programming and data-analysis techniques by transforming raw customer support ticket information into structured and meaningful analytical results.

2. Data Sources

Item	Details
Dataset	Customer support ticket dataset
Domain	Customer Support / Service Operations
Initial Tickets	10
Additional Tickets	3
Final Tickets	13
Data Structure	Dictionary of lists
Tool Used	Google Colab
Programming Language	Python

3. Project Objectives

- Store customer support ticket information using a dictionary of lists.
- Display the initial ticket data in a readable format.
- Add new customer support tickets with automatically generated ticket numbers.
- Validate priority values as High, Medium, or Low.
- Clean and standardize issue descriptions.
- Create a reusable function for keyword-based ticket analysis.
- Analyze selected keywords including poor, good, slow, and excellent.
- Calculate High, Medium, and Low priority ticket counts.
- Identify the longest issue description by word count.
- Extract, count, and sort unique words from cleaned issue descriptions.
- Generate a consolidated final analysis report.

4. Attribute (Column / Feature) Details

Attribute Name	Data Type	Description
Ticket_No	INT	Unique identification number for each support ticket
Customer_Name	STRING	Name of the customer associated with the ticket
Issue_Description	STRING	Description of the customer's support issue
Priority	STRING	Priority assigned to the ticket: High, Medium, or Low

5. Tools & Technologies

Google Colab was used as the development and execution environment.

Python was used for ticket data management, text cleaning, validation, keyword analysis, priority analysis, word counting, set operations, sorting, and final reporting.

GitHub was used for project documentation and repository management.

6. Data Setup & Processing

6.1 Initial Data Loading

- Loaded 10 preloaded customer support tickets.
- Stored ticket fields using a dictionary containing lists.
- Displayed the initial ticket data in a readable format.

6.2 Adding New Tickets

- Collected additional customer name, issue description, and priority information.
- Generated ticket numbers sequentially beginning with Ticket 11.
- Added three new tickets, resulting in a final dataset of 13 tickets.
- Validated priority input using High, Medium, and Low values.

6.3 Text Cleaning

- Converted issue descriptions to lowercase.
- Removed punctuation such as periods, commas, exclamation marks, question marks, and hyphens.
- Removed unnecessary multiple spaces.
- Removed leading and trailing spaces.
- Standardized shorthand such as 'ok' to 'okay'.

7. Python Analysis Structure

7.1 Keyword-Based Ticket Analysis

A reusable function named **count_tickets_with_word()** was created to count the number of ticket descriptions containing a specified keyword. The analysis was performed for four keywords: poor, good, slow, and excellent.

7.2 Priority Analysis

A loop and conditional statements were used to count tickets in the High, Medium, and Low priority categories.

7.3 Longest Issue Analysis

Each cleaned issue description was split into words and its word count was calculated. The ticket with the highest word count was identified.

7.4 Unique Word Analysis

A Python set was used to remove duplicate words from all cleaned issue descriptions. The resulting words were sorted alphabetically and counted.

8. Core Analytical Tasks

Task	Description	Python Concepts Used
T1	Display initial ticket data	Dictionary, lists, loops
T2	Add and validate new tickets	Input, while loop, conditionals
T3	Clean issue descriptions	String methods, split, join
T4	Count selected keywords	Function, loop, string comparison
T5	Analyze priority distribution	Loop, conditional logic

Task	Description	Python Concepts Used
T6	Find longest issue	Loop, split, word count
T7	Extract unique words	Set, loops, sorting
T8	Generate final report	Variables, loops, functions, output formatting

9. Analysis and Key Insights

9.1 Overall Statistics

Metric	Value
Total Tickets	13
High Priority	5
Medium Priority	4
Low Priority	4
Unique Words	34

9.2 Keyword Analysis

Keyword	Ticket Count
Poor	3
Good	3
Slow	3
Excellent	2

9.3 Longest Issue Description

Ticket 11, belonging to Priya, contains the longest cleaned issue description: **"internet connection is slow and poor"**. The description contains **6 words**.

9.4 Unique Word Analysis

The cleaned issue descriptions contain **34 unique words**. The words were extracted using a set and then sorted alphabetically.

10. Insights & Conclusions

10.1 Descriptive Analysis

- Analyzed 13 customer support tickets containing customer, issue, and priority information.
- The final dataset contains 5 High, 4 Medium, and 4 Low priority tickets.
- The selected keywords were analyzed using a reusable Python function.

10.2 Diagnostic Analysis

- The keywords poor, good, and slow each occur in 3 ticket descriptions.
- The keyword excellent occurs in 2 ticket descriptions.
- Ticket 11 has the longest issue description with 6 words.

10.3 Analytical Use

- Priority counts can help organize tickets for support operations.
- Keyword analysis can help identify recurring themes in customer issues.
- Longest-issue analysis demonstrates basic text-length analysis.
- Unique-word extraction provides a simple view of the vocabulary used in support issues.

11. Queries / Code Used

Important: This is a Python project, not a SQL project. Therefore, there are **no SQL queries** in this documentation.

The equivalent analytical work was performed using Python code, including dictionary and list operations, loops, conditional statements, string methods, a reusable keyword-counting function, set operations, sorting, and formatted output.

The project therefore does not require a separate SQL query section. The Python notebook contains the executable analysis steps.

12. Project Deliverables

- Google Colab Python notebook containing the completed analysis.
- One-page project summary.
- GitHub README documentation.
- Project screenshots showing the major analysis stages.
- Final analysis report.

13. Skills Demonstrated

- Python Data Structures – Dictionaries, Lists, and Sets
- Loops – For and While loops
- Conditional Logic – If / Elif / Else
- Function Creation and Reusability
- String Cleaning and Text Processing
- Input Validation
- Keyword-Based Analysis
- Word Counting
- Set Operations and Sorting
- Basic Data Analysis and Reporting
- Project Documentation and GitHub Organization

14. Conclusion

The Customer Support Ticket Analyzer project demonstrates a complete Python-based data analysis workflow. The project covers structured data storage, ticket addition and validation, text cleaning, reusable functions, keyword analysis, priority analysis, longest-issue identification, unique-word extraction, and final reporting.

The analysis transforms raw customer support ticket information into structured results that can be used to understand ticket priorities, recurring keywords, issue length, and the vocabulary present in customer support descriptions.

15. Author

Field	Details
Name	Kumar S
Course	Data Analytics (DA)
Assignment	Module-End Assignment – 4
Tool	Google Colab
Language	Python
Topic	Customer Support Ticket Analyzer